Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Codage des entiers signés, complément à deux, arithmétique binaire et détection de dépassement de capacité (Overflow)

Synthèse exécutive : Ce cours détaille les mécanismes matériels et logiciels utilisés par les processeurs pour représenter et manipuler les nombres entiers relatifs (positifs et négatifs) en mémoire. En l'absence de composant électronique capable de stocker directement un signe '+' ou '-', l'architecture des ordinateurs repose sur la convention du complément à deux sur un nombre fixe de N bits. Cette méthode élimine le problème du double zéro (présent dans le codage Signe et Magnitude) et permet d'utiliser le même circuit électronique d'addition dans l'Unité Arithmétique et Logique (ALU) pour effectuer à la fois des additions et des soustractions. Le cours couvre également la méthode rigoureuse de conversion, le calcul de la plage de valeurs possibles, ainsi que la différenciation fondamentale entre la retenue sortante (Carry out) et le dépassement de capacité (Overflow).

1. Problématique matérielle et justification du Complément à Deux

Au niveau le plus bas de l'architecture informatique, l'information est stockée dans des transistors et des bascules bistables ne reconnaissant que deux états logiques (0 et 1). Un ordinateur ne possède aucun moyen d'inscrire directement un glyphe comme '-' ou '+'. Pour représenter des entiers relatifs, les concepteurs d'architectures ont d'abord envisagé le codage 'Signe + Magnitude' (où le bit le plus à gauche indique le signe et les autres la valeur absolue). Cependant, ce codage génère deux répresentations pour zéro (+0 et -0), ce qui gaspille une combinaison binaire et complexifie les tests d'égalité. De plus, il impose d'ajouter un circuit de soustraction distinct dans l'ALU. Le complément à deux résout totalement ces problèmes : zéro possède une représentation unique (0000...0) et l'opération de soustraction A - B devient une simple addition A + (-B), réutilisant le circuit d'addition binaire existant.

- Absence de signe physique : les bascules électroniques des registres ne stockent que les états 0 (0V) ou 1 (Vcc).
- Échec de la représentation Signe et Magnitude : présence d'un double zéro (+0 et -0) et besoin de circuits séparés pour l'addition et la soustraction.
- Unification par le complément à deux : la soustraction A - B s'effectue comme A + (-B) avec le même additionneur au niveau de l'ALU.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur est bloqué à 0000 km et qu'on effectue un kilomètre en marche arrière, le compteur affiche 9999. En binaire sur 8 bits, retrancher 1 à 00000000 donne 11111111_2, qui définit la représentation de -1 en complément à deux.*

2. Algorithme de Conversion et Plage de Valeurs sur N bits

Pour convertir un nombre négatif -X en complément à deux sur un mot mémoire de N bits, il faut suivre une méthode stricte en trois étapes. Premièrement, on écrit la valeur absolue X en binaire pur sur N bits complets (en rajoutant les 0 nécessaires à gauche). Deuxièmement, on applique le complément à un en inversant chaque bit (les 0 deviennent 1, les 1 deviennent 0). Troisièmement, on ajoute 1 au résultat binaire. La plage des nombres représentables sur N bits n'est pas symétrique : sur 8 bits par exemple, elle s'étend de -128 à +127. Cette dissymétrie provient du fait que le zéro est inclus dans le domaine positif (MSB = 0).

- Algorithme des 3 étapes : Valeur absolue sur N bits -> Inversion des bits (Complément à 1) -> Addition de 1 (Complément à 2).
- Rôle du bit MSB (Most Significant Bit) : le bit le plus à gauche indique le signe (0 pour positif, 1 pour négatif).
- Dissymétrie de la plage de valeurs : de -2^(N-1) jusqu'à +(2^(N-1) - 1).

Cas concret enseignant : *Avertissement crucial du professeur pour les partiels : il faut impérativement fixer la taille de N bits (ex: 8 bits) AVANT d'effectuer l'inversion des bits. Inverser '1100' sans mettre au préalable les zéro de gauche '00001100' produit un résultat complètement faux.*

3. Arithmétique, Retenue (Carry) et Détection du Dépassement de Capacité (Overflow)

En travaillant sur des registres de taille fixe (ex: 8, 16, 32 ou 64 bits), les calculs arithmétiques peuvent dépasser la capacité maximale du registre. Il est vital de distinguer le Carry Out (retenue sortant du bit le plus à gauche) du drapeau d'Overflow (OV). Lors d'une addition en complément à deux sur N bits, le Carry Out est simplement ignoré car le calcul s'effectue modulo 2^N. L'Overflow, en revanche, signale que le résultat dépasse la plage représentable (-2^(N-1) à +2^(N-1)-1). La règle formelle de détection d'un Overflow en examen est : un Overflow survient si et seulement si deux nombres de même signe additionnés produisent un résultat de signe inverse. L'addition de deux nombres de signes différents ne peut jamais générer d'Overflow.

- Soustraction unifiée : calculée sous la forme A + (-B).
- Distinction Carry vs Overflow : le Carry (retenue sortante du MSB) est ignoré en binaire signé, tandis que l'Overflow indique une erreur de calcul arithmétique.
- Règle d'or de l'Overflow : un dépassement se produit uniquement lors de l'addition de deux opérandes de même signe dont le résultat est de signe opposé.

Cas concret enseignant : *L'anecdote historique du crash d'Ariane 5 (vol 501 en 1996) : un nombre flottant sur 64 bits a été converti en entier signé 16 bits sans protection contre le dépassement de capacité. L'Overflow non géré a provoqué la défaillance du système de guidage et la destruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : Parce que le registre du processeur a une taille fixe de N bits (arithmétique modulo 2^N). Le complément à deux est mathématiquement conçu pour que la retenue sortante dépasse la capacité du registre et soit ignorée, laissant sur les N bits restants exactement la valeur correcte de la somme.

Question étudiant : Comment l'ordinateur fait-il la différence entre le nombre binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : Au niveau matériel, l'ordinateur ne fait aucune différence : ce sont juste 8 bits à 1 stockés en mémoire. C'est l'instruction du programme (le type de donnée choisi par le développeur, comme uint8_t ou int8_t en C) qui indique à l'ALU quelle instruction d'interprétation ou de comparaison appliquer.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler l'écriture binaire des opérandes, effectuer l'addition binaire correspondante, donner le résultat décimal et indiquer s'il y a eu un Overflow.

Résolution pas à pas :
1. Conversion de +35 sur 8 bits : 35 = 32 + 2 + 1 = 00100011_2. 2. Conversion de -75 en complément à deux sur 8 bits : - Valeur absolue +75 sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (-75 décimal). 3. Pose de l'addition (+35) + (-75) : 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 4. Interprétation du résultat : Le bit MSB est à 1, le résultat est donc négatif. Pour retrouver sa valeur absolue, on applique le complément à deux sur 11011000_2 : - Inversion : 00100111_2 - Ajout de 1 : 00101000_2 = 32 + 8 = 40 décimal. Le résultat binaire 11011000_2 représente donc -40 en décimal. 5. Analyse de l'Overflow : On additionne un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). L'addition de deux nombres de signes opposés ne peut jamais produire d'Overflow.

Résultat : Résultat binaire : 11011000_2 (-40 en décimal). Drapeau d'Overflow = 0. Le calcul 35 - 75 = -40 est exact et valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage des entiers signés représentables sur un mot de 16 bits en complément à deux ?

R : La plage s'étend de -2^(16-1) à +(2^(16-1) - 1), c'est-à-dire de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine un Overflow lors de l'addition de deux entiers signés ?

R : Un Overflow se produit si et seulement si on additionne deux opérandes de même signe (deux positifs ou deux négatifs) et que le résultat obtenu possède un signe opposé.

Q : Quel est le résultat de l'inversion et de l'ajout de 1 (complément à deux) appliqué au nombre zéro (00000000_2) sur 8 bits ?

R : Inversion de 00000000 donne 11111111. Ajout de 1 donne (1)00000000. Sur 8 bits, le résultat est 00000000_2, ce qui prouve l'unicité de la représentation du zéro.